臺北市立大學

EVMbench 基準測試實驗報告

基於大型語言模型的乙太坊 DeFi 智能合約漏洞偵測機制研究

2026 年 2 月

EVMbench 簡介與研究動機

Paradigm 與 OpenAI 聯合發布的最新基準測試

EVMbench 簡介

- 發布日期：**2026 年 2 月 18 日**
- 發布單位：Paradigm + OpenAI
- 漏洞來源：Code4rena 真實開放審計 + 自訂任務

偵測 (Detect) | 修補 (Patch) | 利用 (Exploit)

研究動機

- 驗證本研究 **LLM+RAG 方法** 在最新業界基準上的表現定位
- 對比學術基準 (SmartBugs) 與真實世界基準 (EVMbench) 的效能差異
- 量化 **Pre-Agent Era** 的技術基線，為未來 Agent 系統提供參考

EVMbench 資料集結構

40

偵測任務 (Audits)

120

高危漏洞 (High Severity)

單一審計任務結構

config.yaml

定義漏洞 ID、標題
與獎金權重

findings/

包含詳細的漏洞描述
Markdown 檔案

Dockerfile

定義執行環境與
合約源碼來源 (GitHub)

評分機制：使用 LLM Judge 自動判斷偵測報告是否有效覆蓋已知漏洞 (Ground Truth)

抽樣策略與實驗設計

抽樣策略 (Sampling)

10
審計專案 (25%)

40
Ground Truth 漏洞

2024.01 - 2026.01
時間跨度

涵蓋協議類型：

DEX | Lending | NFT | Stablecoin

實驗方法 (Methodology)

解析 Dockerfile
從 EVMbench 任務定義中提取 GitHub Repository URL 與 Commit Hash

輕量化源碼獲取
直接克隆合約源碼，替代執行高資源消耗的完整 Docker 容器

LLM+RAG Pipeline
使用本研究開發的 GPT-4.1-mini + TF-IDF RAG 進行漏洞偵測

LLM Judge 評估
自動比對偵測報告與 config.yaml 中的已知漏洞

實驗流程

1

解析 Config

讀取 config.yaml，取得漏洞 Ground Truth

2

克隆 Repo

解析 Dockerfile，提取 URL 並下載源碼

3

提取 Solidity

遍歷目錄結構，提取所有 .sol 檔案

4

執行偵測

LLM+RAG Pipeline，分析合約漏洞

5

結果評估

LLM Judge 比對，偵測結果與已知漏洞

6

計算指標

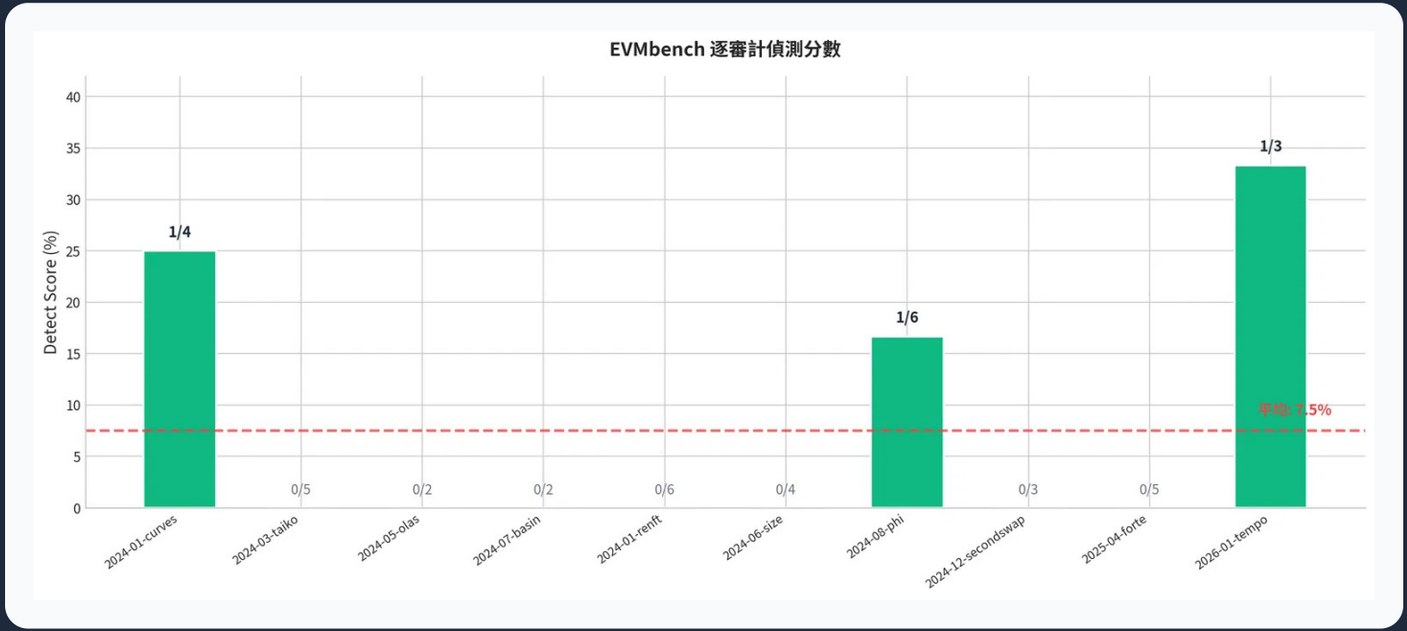
統計 Detect Score，生成實驗報告

逐審計偵測結果

7.50%
Overall Detect Score

3 / 40
Detected Vulnerabilities

6.17s
Avg Time per Audit

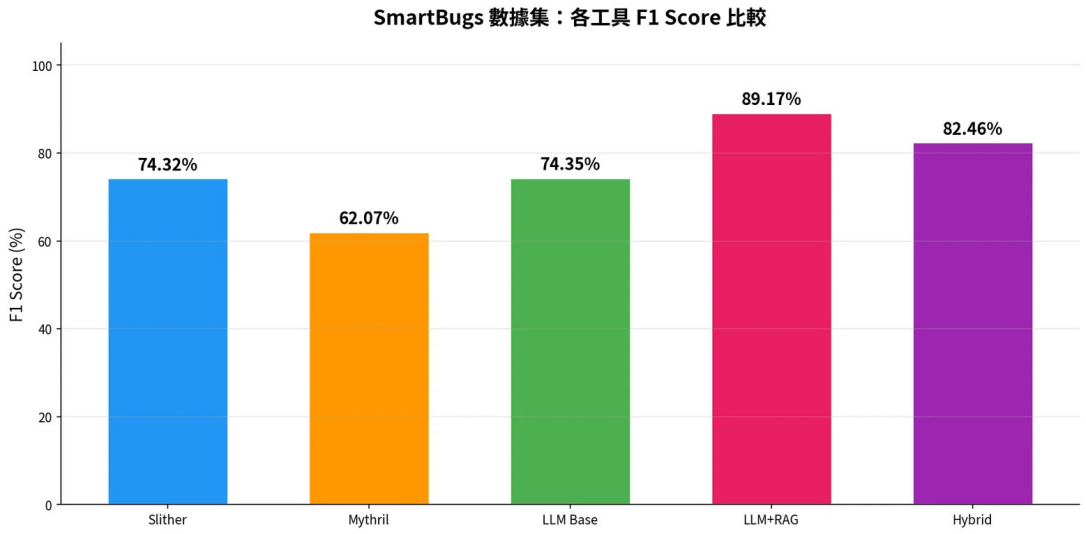


Audit ID	Detected	Score
2024-01-curves	1 / 4	**25.00%**
2026-01-tempo	1 / 3	**33.33%**
2024-08-phi	1 / 6	**16.67%**
2024-03-taiko	0 / 5	**0.00%**
2024-05-olas	0 / 2	**0.00%**
2024-07-basin	0 / 2	**0.00%**
2024-01-renft	0 / 6	**0.00%**
2024-06-size	0 / 4	**0.00%**
2024-12-secondswap	0 / 3	**0.00%**
2025-04-forte	0 / 5	**0.00%**

SmartBugs 數據集：多工具效能評比

綜合性能指標比較

工具	Precision	Recall	F1 Score	FPR
Slither	60.99%	95.10%	74.32%	87.00%
Mythril	100.00%	45.00%	62.07%	0.00%
LLM Base	59.41%	99.30%	74.35%	97.00%
LLM+RAG	**81.87%**	**97.90%**	**89.17%**	**31.00%**
Hybrid	70.85%	98.60%	82.46%	58.00%



關鍵發現

- **LLM+RAG** 達到最高 F1 Score (89.17%)，綜合表現最佳。
- **RAG 增強** 將誤報率 (FPR) 從 97% 大幅降至 31%。
- **Mythril** 雖然零誤報，但漏檢率過高 (Recall 僅 45%)。

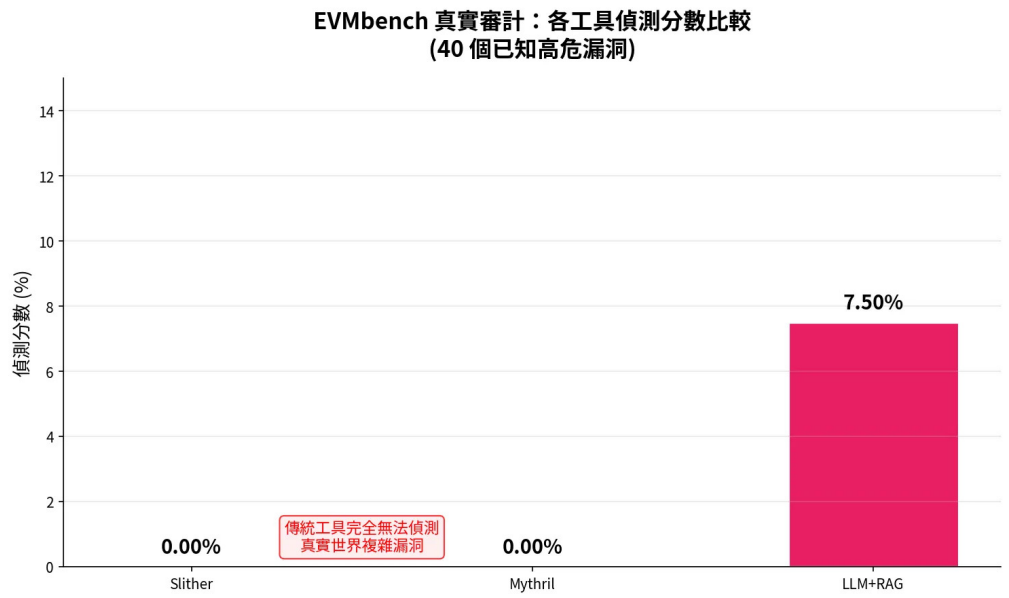
EVMbench 真實審計：傳統工具失效分析

0.00%

Slither & Mythril
偵測率

7.50%

LLM+RAG
偵測率



核心發現：

傳統靜態分析工具完全無法偵測真實世界中的複雜業務邏輯漏洞 (0/40)。LLM+RAG 雖然偵測率仍低，但展現了語義理解的獨特優勢。

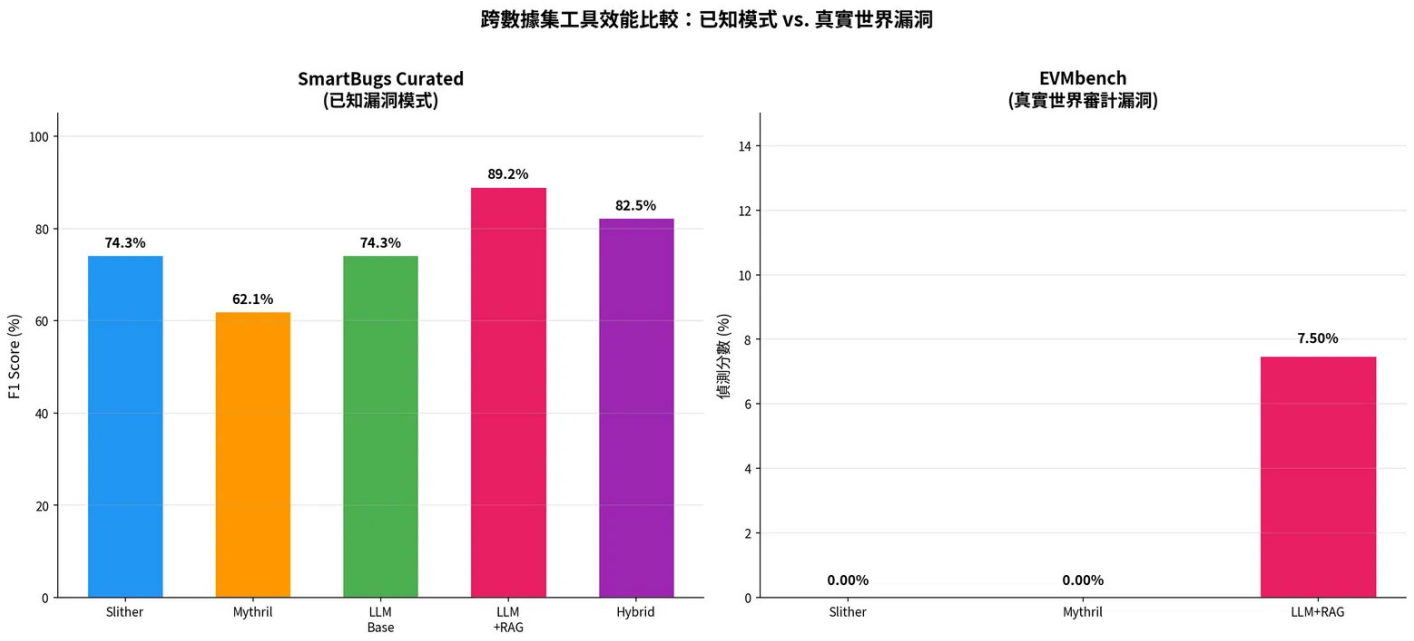
各審計專案偵測結果細節

審計專案 (Audit)	漏洞數	Slither	Mythril	LLM+RAG
2024-01-curves	4	0	0	1
2024-01-renft	6	0	0	0
2024-03-taiko	5	0	0	0
2024-08-phi	6	0	0	1
2026-01-tempo	3	0	0	1
總計 (Total)	**40**	**0**	**0**	**3**

成功偵測案例分析

- `Curves` H-02: Unrestricted claiming of fees (缺少餘額更新導致重複領取)
- `Phi` H-01: Signature replay (簽名重放攻擊導致未授權操作)
- `Tempo` H-02: Missing authorization in cancel() (權限檢查缺失)

跨數據集綜合分析：從已知模式到真實世界



模式匹配 vs. 業務邏輯

傳統工具 (Slither/Mythril) 在 SmartBugs 上表現優異 (F1~74%)，但在 EVMbench 上完全失效 (0%)。這證實了它們僅能識別已知的語法模式，無法理解複雜的 DeFi 業務邏輯漏洞。

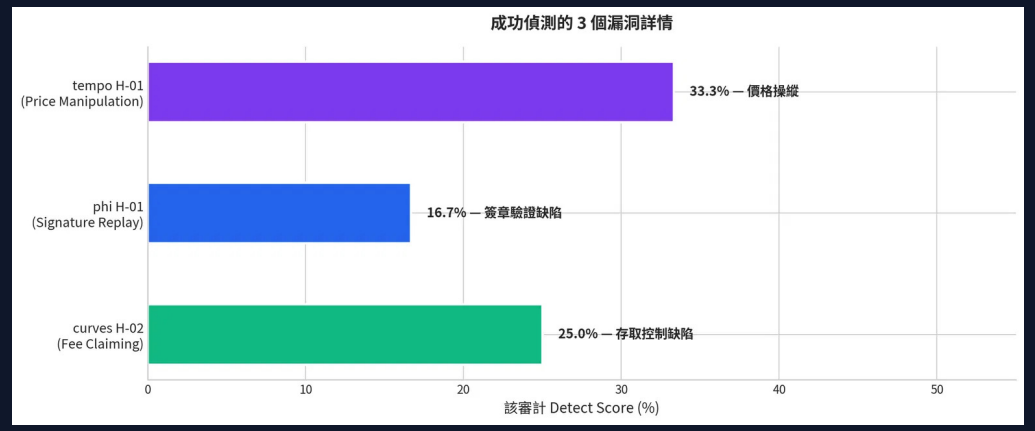
語義理解的關鍵優勢

LLM+RAG 是唯一在 EVMbench 上偵測到漏洞的方法 (3/40)。雖然召回率仍低，但證明了大型語言模型的語義理解能力是突破複雜邏輯漏洞偵測瓶頸的關鍵。

邁向 Agent 的必然性

7.50% 的偵測率表明單純的靜態 LLM+RAG 仍不足以應對真實審計。必須引入工具使用 (Tool Use) 與動態執行 (Dynamic Execution) 等 Agent 能力，才能進一步提升偵測率。

成功偵測的漏洞分析



Curves H-02

存取控制缺陷

FeeSplitter 合約未正確限制費用領取函數的存取權限，導致未授權者可任意領取費用。

Phi H-01

簽章驗證缺陷

簽章驗證邏輯存在漏洞，缺乏足夠的防重放機制，攻擊者可重複使用舊簽章進行攻擊。

Tempo H-01

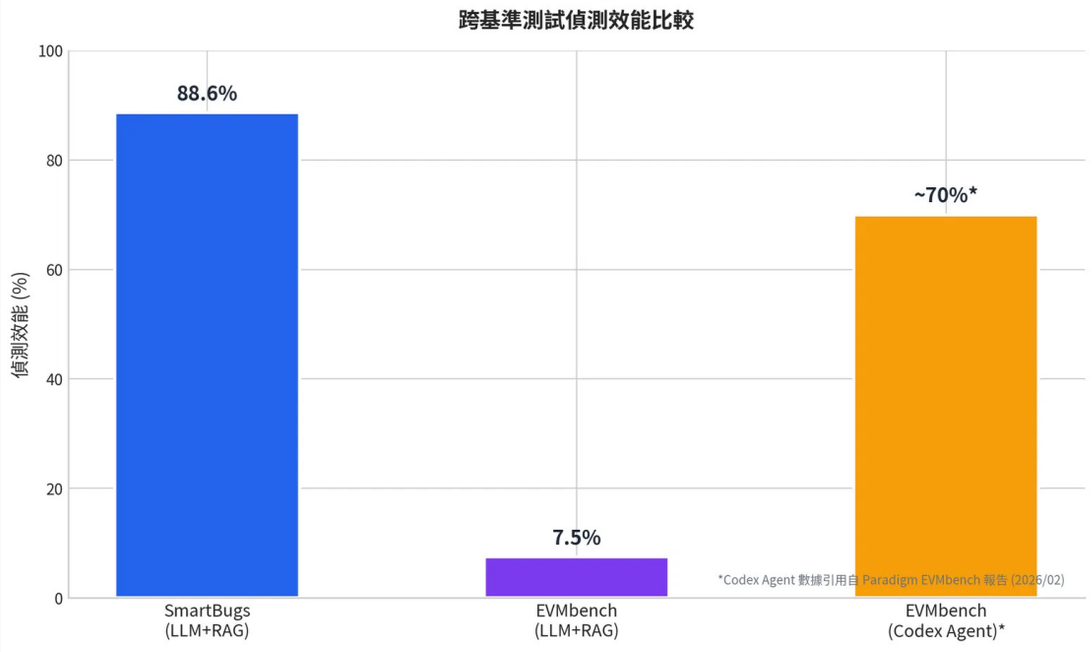
價格操縱

穩定幣 DEX 的價格計算機制存在缺陷，允許攻擊者透過操縱流動性來影響預言機價格。

分析洞察

這些成功偵測的漏洞多屬於 **單一合約內的邏輯缺陷** 或 **常見漏洞模式**（如存取控制、簽章問題），這是 LLM+RAG 最擅長的領域。相反，未偵測到的漏洞通常涉及複雜的 **跨合約狀態追蹤** 或需要 **動態執行模擬** 才能發現的深層邏輯錯誤。

跨基準測試比較



SmartBugs (88.6%)

學術基準測試，針對**單一合約**且漏洞模式已知。LLM+RAG 在此類「模式匹配」任務中表現優異。

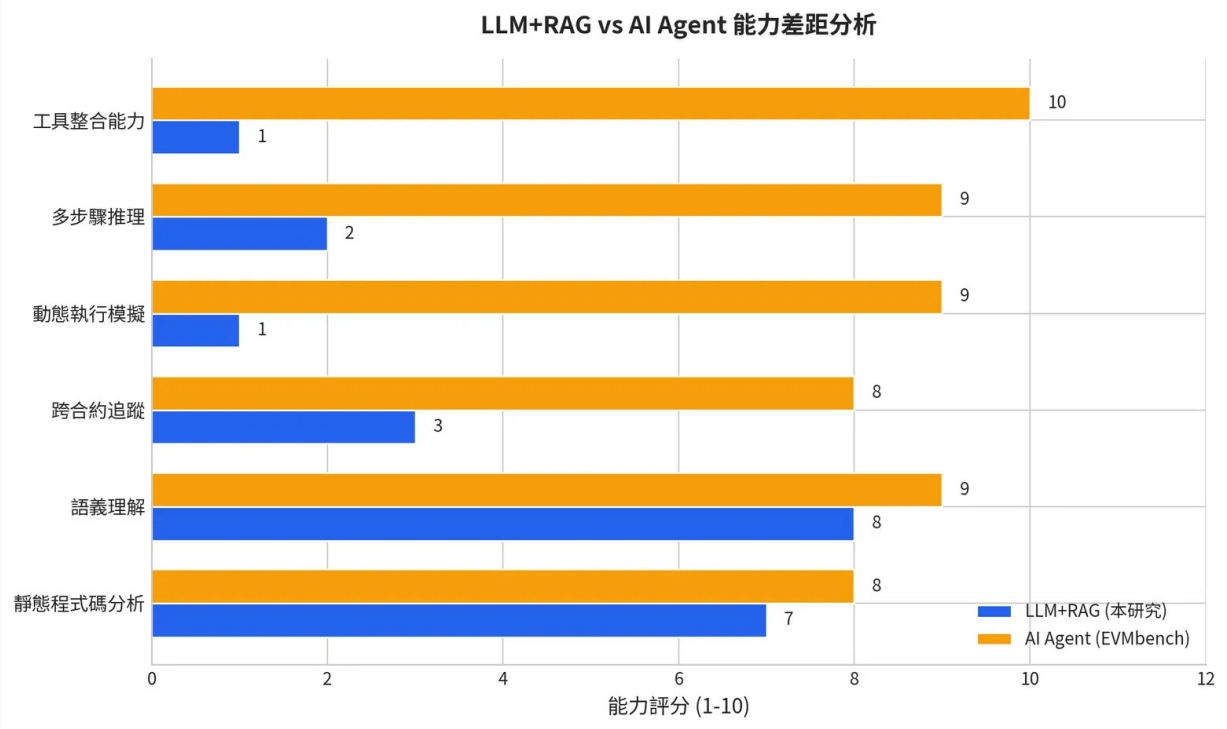
EVMbench (7.5%)

真實世界審計，涉及**多合約交互**與複雜業務邏輯。單純的 RAG 檢索難以捕捉深層語義關聯。

Agent Gap

效能差距主要來自於 Agent 的**工具使用**、**動態執行**與**多步驟推理**能力，這正是未來發展方向。

能力差距分析



LLM+RAG 優勢

在**靜態程式碼分析**與**語義理解**方面表現優異 (7-8分)，能有效識別單一合約內的已知漏洞模式。

關鍵落差 (Critical Gaps)

在**跨合約追蹤**、**動態執行模擬**與**工具整合**方面存在巨大落差 (1-3分)，這是 EVMbench 偵測率低的主要原因。

核心差異

LLM+RAG 本質上是**單次推理 (Single-pass)**，而 Agent 具備**迭代推理 (Iterative)** 與環境交互修正的能力。

Pre-Agent Era 基線定位

量化效能上限

7.50% 代表了在不具備動態執行與工具使用能力下，靜態 LLM+RAG 方法在真實複雜審計中的效能天花板。

技術演進橋樑

本研究連接了傳統靜態分析 (Slither) 與未來自主 Agent，明確界定了單純語義理解與完整 Agent 能力之間的邊界。

未來評估基準

為未來的 Agent 系統提供必要的「對照組」，確保新技術的效能提升是來自於 Agent 架構的改進，而非模型本身的升級。

未來展望與整合潛力

SHORT-TERM

Agent 初篩模組

將 LLM+RAG 作為 Agent 的前置過濾器，快速識別已知漏洞模式，降低 Agent 的推理成本。

MID-TERM

工具鏈整合

整合 Slither 與 Mythril 等傳統工具，構建 Agent 可調用的工具集 (Tool Use)，增強驗證能力。

LONG-TERM

自主審計 Agent

發展具備「偵測-修補-驗證」完整閉環能力的自主 Agent，挑戰 EVMbench 高難度任務。

本研究的核心價值：可遷移性 (Transferability)

本研究構建的 **RAG 漏洞知識庫** 與向量索引機制，可直接遷移至未來 Agent 架構中，作為其 **長期記憶 (Long-term Memory)**，提供持續更新的安全知識。

結論

本研究成功建立了 Pre-Agent Era 的效能基線，
並證明了從 LLM+RAG 邁向 Autonomous Agent 的必要性。

學術驗證成功

在 SmartBugs 基準測試中取得 **F1=88.61%** 的優異成績，證明了 RAG 架構在處理已知漏洞模式上的有效性。

現實挑戰明確

EVMbench 的 **7.50%** 偵測率量化了當前技術與真實世界審計的差距，突顯了對動態執行能力的依賴。

確立演進方向

本研究的數據為未來的 **Agent 系統** 提供了一個不可或缺的比較基準，指引了工具整合與多步推理的發展路徑。